

# Домашнее задание 1: подготовка стенда на macOS

Выполняйте эту инструкцию **после** занятия «Семинар 1. Настройка S3 и колоночные форматы: подготовка источника данных», как этап 0 домашней работы. На семинаре команды выполняет преподаватель; студентам заранее устанавливать или запускать стенд не нужно.

Здесь вы установите зависимости и проверите доступность среды. После этого переходите к [воспроизведению учебного примера в ДЗ](#), затем — к своему открытому источнику.

## Что будет запущено

Docker Compose поднимает MinIO, Spark 3.5.9, PostgreSQL 16 и Trino 483. Spark использует библиотеку Iceberg 1.11.0; отдельного сервиса Iceberg нет. Java и Spark на хост устанавливать не нужно — они находятся в контейнерах. Python 3 нужен только для генерации учебного CSV.

Docker-контейнер — изолированный процесс с упакованными зависимостями; образ — шаблон для его запуска. Compose читает [docker-compose.yml](#) и поднимает связанные сервисы. Volume — постоянное хранилище контейнера, которое переживает обычную остановку.

Стенд учебный и локальный. В Compose опубликованные порты привязаны к [127.0.0.1](#), Trino ограничен 2 ГБ памяти. Не публикуйте его порты в интернет.

## Требования

- macOS, поддерживаемая текущим Docker Desktop: текущий и два предыдущих major-релиза по официальной политике Docker;
- Apple Silicon или Intel Mac;
- аппаратная виртуализация;
- минимум 8 ГБ RAM, для комфортной работы рекомендуется 12–16 ГБ;
- 12–15 ГБ свободного места на первый build и данные;
- стабильный интернет для загрузки образов и Maven-зависимостей.

Docker указывает 4 ГБ как базовый минимум продукта, но учебный стек одновременно запускает несколько JVM/сервисов, поэтому 8 ГБ — ориентир для этого стенда. Если ноутбук не позволяет выделить указанные ресурсы, обсудите с преподавателем доступную учебную машину или иной способ выполнения. Работа остаётся индивидуальной.

## 1. Установите Docker Desktop

Скачайте вариант для своего процессора: [Docker Desktop for Mac](#).

Проверьте архитектуру:

```
uname -m
```

- `arm64` — Apple Silicon;
- `x86_64` — Intel.

После установки запустите Docker Desktop и дождитесь статуса Running. Rosetta не является безусловным требованием всего стенда, но Docker рекомендует её для отдельных инструментов командной строки на Apple Silicon; следуйте подсказке установщика, если она появится.

Проверка:

```
docker version
docker compose version
docker run --rm hello-world
```

## 2. Проверьте Python 3

macOS не гарантирует наличие подходящего Python 3 «из коробки».

```
python3 --version
```

Если команды нет, установите Python с [python.org](https://python.org) или через уже установленный Homebrew:

```
brew install python
```

Homebrew не требуется, если Python установлен другим способом.

## 3. Выделите ресурсы Docker

Docker Desktop → Settings → Resources:

- CPU: 4 cores, если доступны;
- Memory: 6 ГБ минимум для этого стенда, 8 ГБ комфортно;
- Disk image: убедитесь, что свободно не меньше 12 ГБ.

Не отдавайте Docker почти всю память ноутбука: macOS тоже нужен запас.

## 4. Подготовьте каталог стенда

Скачайте `lecture_01_student.zip` по ссылке преподавателя и распакуйте его. В папке `lecture_01_student/` находятся инструкции, ДЗ и `infra/`. Репозиторий преподавателя не нужен. Замените пример пути на путь к распакованной папке; кавычки сохраняйте, если в пути есть пробелы. Все дальнейшие команды, кроме проверки занятых портов, выполняйте из `infra/`.

```
cd "/путь/к/lecture_01_student/infra"
```

Проверка конфигурации без запуска:

```
docker compose config --quiet
```

## 5. Соберите образы

```
docker compose build
```

Первый build скачивает крупные зависимости и может занять заметное время. Не переходите к запуску сервисов, пока команда не завершится успешно. Повторный build обычно использует кэш.

## 6. Запустите сервисы

```
docker compose up -d
docker compose ps --all
```

Дождитесь готовности PostgreSQL и MinIO. Сервис `minio-init` создаёт бакеты и завершается: статус `Exited (0)` для него нормален. В студенческом комплекте имя проекта — `hse-lakehouse-student`; имена контейнеров Compose создаёт автоматически. Остальные четыре сервиса должны оставаться запущенными; `Exited` с ненулевым кодом или постоянный `Restarting` требуют диагностики ниже.

Минимальные проверки среды:

```
docker compose exec spark python3 -c "import pyspark; print(pyspark.__version__)"
docker compose exec trino trino --execute "SHOW CATALOGS"
```

Версия PySpark должна совпасть с версией в `infra/spark/Dockerfile` (сейчас 3.5.9); в Trino должны быть `lakehouse` и `memory`. Если Trino ещё запускается, проверьте его логи и повторите запрос после сообщения о готовности.

Интерфейсы:

| Сервис        | Адрес                              | Доступ                            |
|---------------|------------------------------------|-----------------------------------|
| MinIO Console | <code>http://localhost:9001</code> | <code>admin / hse2026minio</code> |
| MinIO S3 API  | <code>http://localhost:9000</code> | для скриптов                      |
| Trino UI      | <code>http://localhost:8088</code> | без пароля, только локально       |

Откройте MinIO Console и убедитесь, что есть бакеты `raw` и `datalake`. На этом проверка установки закончена: данные и таблицы ещё не создавались. Перейдите к [этапу 0 Д3](#) и выполните последовательность учебного примера один раз. Она включает генерацию небольшого CSV на 200 000 строк; отдельный большой синтетический набор не нужен.

## SQL-клиент для занятия и домашней работы

На семинаре преподаватель работает с Trino через DataGrip. Подключение описано в [отдельной инструкции](#): `localhost:8088`, пользователь `teacher`, пароль пустой, каталог `lakehouse`, схема `dwh`. Встроенный сайт Trino показывает выполнение запросов; DataGrip предоставляет SQL-редактор и табличный результат. В DataGrip можно просматривать и выполнять сохранённый `infra/trino/scripts/seminar_demo.sql` по блокам.

Для Д3 DataGrip необязателен: тот же SQL выполняется через CLI. Выбор клиента не влияет на 10-балльную оценку. Устанавливать новый клиент во время семинара не требуется. После перезапуска Trino восстановите справочник из блока 5 SQL-файла.

Preview CSV, JSON и Parquet в установленной MinIO Console не поддерживается. В MinIO проверяйте ключ, размер и список объектов; CSV/JSON открывайте в редакторе, а строки Parquet/Iceberg показывайте через читающий их движок.

Автозапуск учебных контейнеров в Compose отключён (`restart: "no"`). После запуска Docker Desktop вручную выполните `docker compose up -d` из `infra/`.

## Диагностика

```
docker compose ps --all
docker compose logs --tail=100 postgres
docker compose logs --tail=100 minio
docker compose logs --tail=100 minio-init
docker compose logs --tail=100 spark
docker compose logs --tail=100 trino
```

Порт занят:

```
lsof -nP -iTCP:9000 -sTCP:LISTEN
lsof -nP -iTCP:9001 -sTCP:LISTEN
lsof -nP -iTCP:8088 -sTCP:LISTEN
```

Apple Silicon: если конкретный образ запускается через эмуляцию и падает, сначала обновите Docker Desktop. Не добавляйте `platform: linux/amd64` без ошибки, подтверждающей необходимость — это замедляет стек.

## Остановка и очистка

Остановить, сохранив volumes:

```
docker compose down
```

Удалить учебные данные и начать с чистого состояния:

```
docker compose down -v
```

Команда с `-v` удаляет volumes MinIO и PostgreSQL без возможности восстановления. Используйте её только для этого учебного стенда.

## Источники

- [Docker Desktop for Mac — установка и требования](#)
- [Docker Desktop release notes](#)
- [Apache Spark releases](#)
- [Apache Iceberg releases](#)
- [Trino releases](#)